

Asymptotic notations and Algorithm efficiency

Analysis of non-recursive and
recursive algorithms

Asymptotic analysis for
recurrence relation: Iteration
method, Substitution method,
Recursive tree method

Big-O

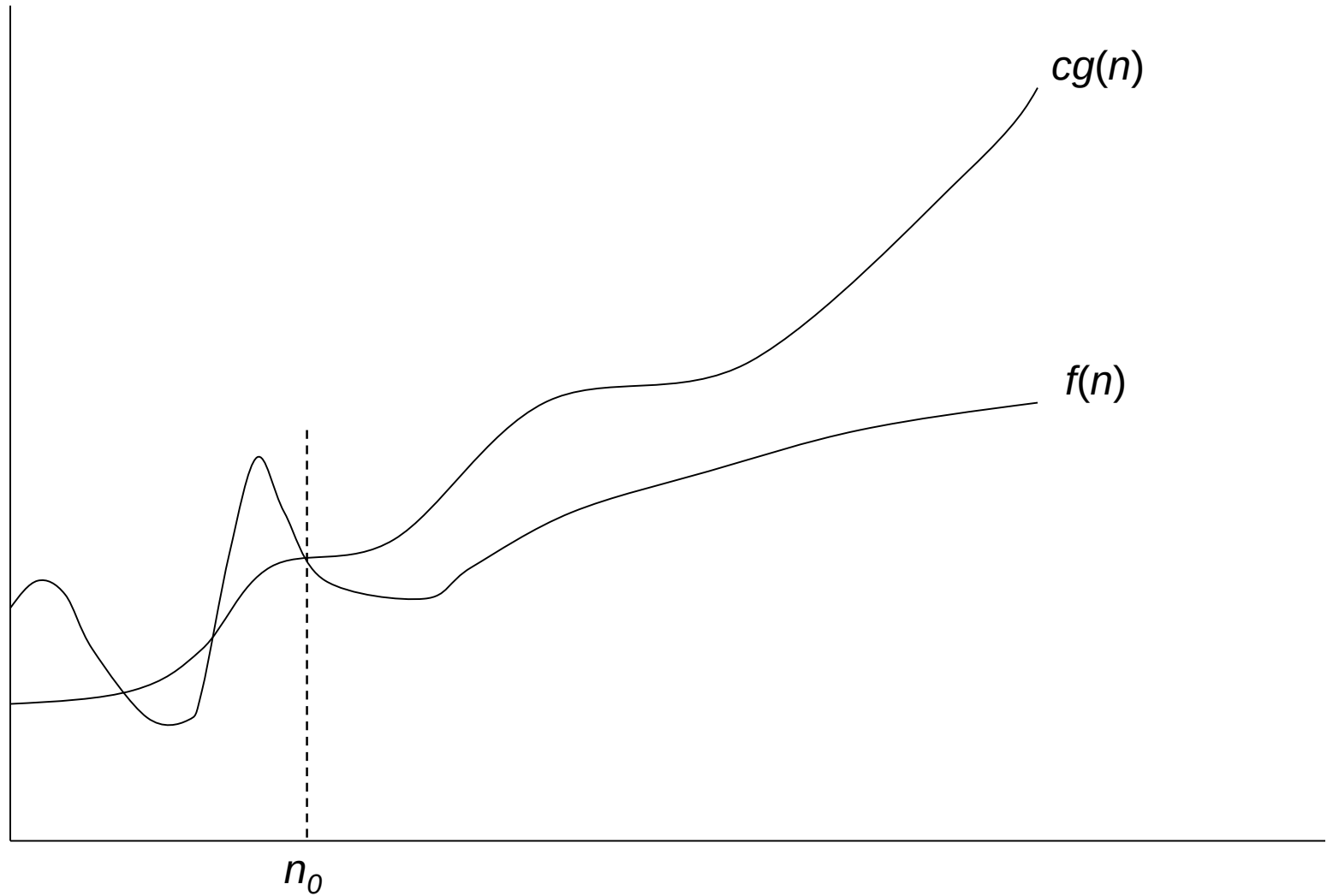
$f(n) = O(g(n))$: there exist positive constants c and n_0 such that
$$0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$

- What does it mean?

- If $f(n) = O(n^2)$, then:

- $f(n)$ can be larger than n^2 sometimes, **but...**
 - We can choose some constant **c** and some value **n_0** such that for **every** value of **n** larger than **n_0** : **$f(n) < cn^2$**
 - That is, for values larger than n_0 , $f(n)$ is never more than a constant multiplier greater than n^2
 - Or, in other words, $f(n)$ does not grow more than a constant factor faster than n^2 .

Visualization of $O(g(n))$



Examples

✓ $2n^2 = O(n^3)$: $2n^2 \leq cn^3 \Rightarrow 2 \leq cn \Rightarrow c = 1 \text{ and } n_0 = 2$

✓ $n^2 = O(n^2)$: $n^2 \leq cn^2 \Rightarrow c \geq 1 \Rightarrow c = 1 \text{ and } n_0 = 1$

✓ $n = O(n^2)$: $n \leq cn^2 \Rightarrow cn \geq 1 \Rightarrow c = 1 \text{ and } n_0 = 1$

Tight bounds

- We generally want the tightest bound we can find.

Big Omega – Notation

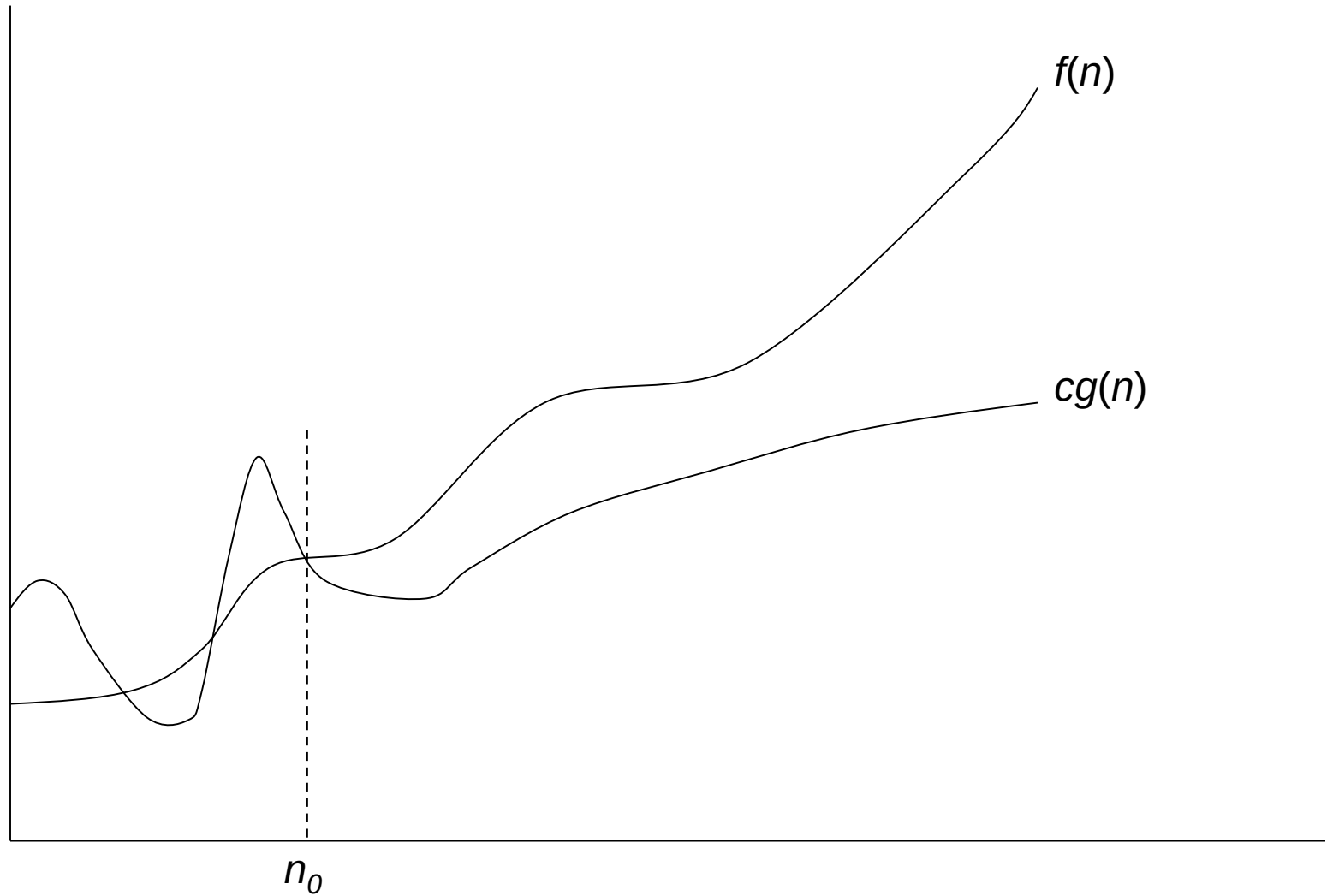
- $\Omega()$ – A **lower** bound

$f(n) = \Omega(g(n))$: there exist positive constants c and n_0 such that

$$0 \leq f(n) \geq cg(n) \text{ for all } n \geq n_0$$

- $n^2 = \Omega(n)$
- Let $c = 1$, $n_0 = 2$
- For all $n \geq 2$, $n^2 > 1 \times n$

Visualization of $\Omega(g(n))$

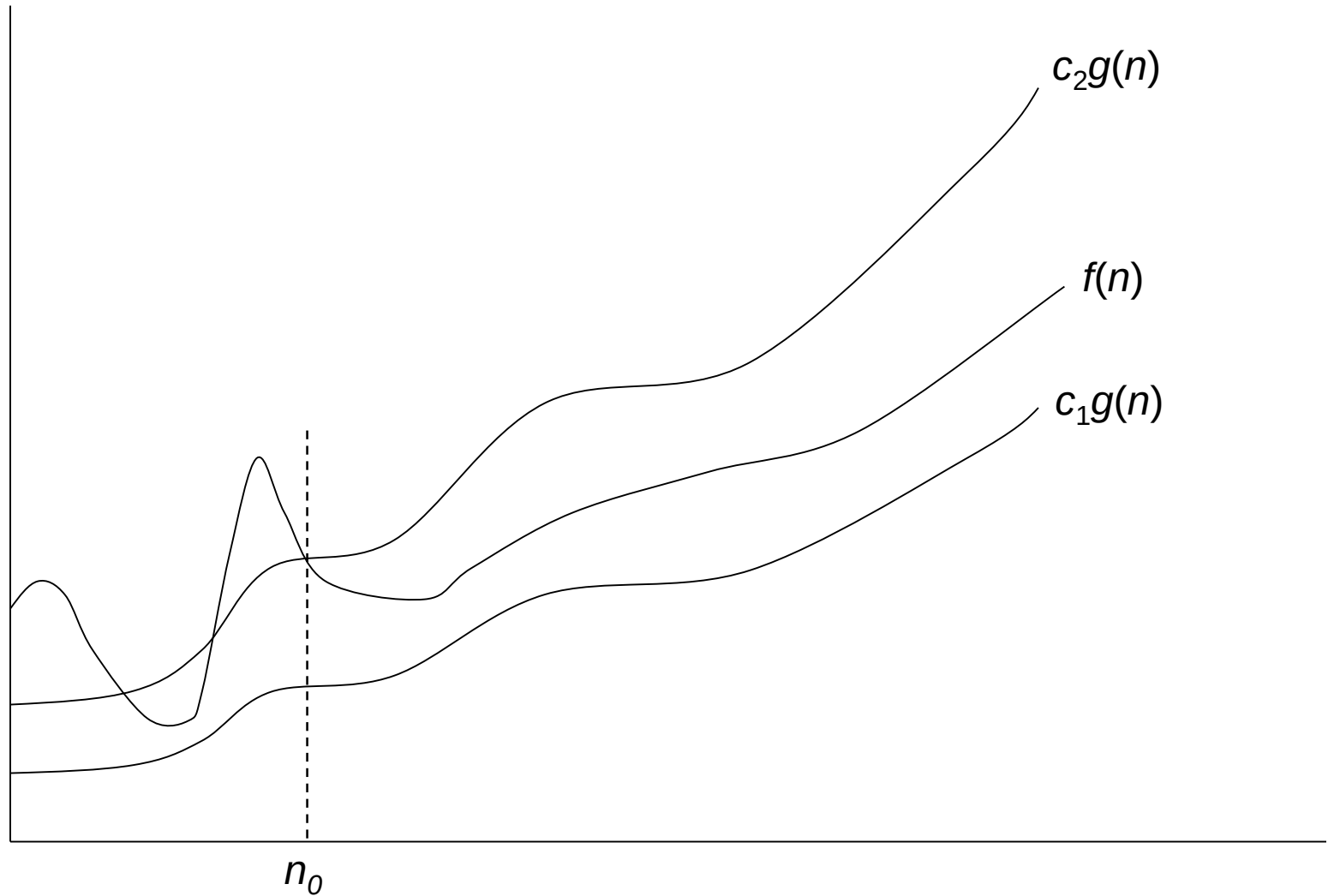


Θ -notation

$f(n) = \Theta(g(n))$: there exist positive constants c_1, c_2 , and n_0 such that
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = O(g(n)) \text{ AND } f(n) = \Omega(g(n))$$

Visualization of $\Theta(g(n))$



Sequential Search

- Given an **unsorted** vector/list $a[]$, find the location of element X .

```
for (i = 0; i < n; i++) {  
    if (a[i] == X) return true;  
}  
return false;
```

- Input size: $n = \text{array size}()$
- Complexity = $O(n)$ Worst case
- $\Omega(1)$ Best case
- $\theta(n/2) \Rightarrow \theta(n)$ Average case
-

If-then-else Statement

```
if(condition)
    i = 0;
else
    for ( j = 0; j < n; j++)
        a[j] = j;
```

- Complexity = ??
= $O(1) + \max (O(1), O(N))$
= $O(1) + O(N)$
= $O(N)$

Consecutive Statements

- Add the complexity of consecutive statements

```
for (j = 0; j < n; ++j) {  
    // 3 atomics  
}  
for (j = 0; j < n; ++j) {  
    // 5 atomics  
}
```

- Complexity = $O(3n + 5n) = O(n)$

Nested Loop Statements

- Analyze such statements inside out

```
for (j = 0; j < n; ++j) {  
    // 2 atomics  
    for (k = 0; k < n; ++k) {  
        // 3 atomics  
    }  
}
```

- Complexity = $O((2 + 3n)n) = O(n^2)$

Recursion

```
long factorial( int n )
{
    if( n <= 1 )
        return 1;
    else
        return n*factorial(n- 1);
}
```

In terms of big-Oh:

$$t(1) = 1$$

$$t(n) = 1 + t(n-1) = 1 + 1 + t(n-2) \\ = \dots k + t(n-k)$$

Choose $k = n-1$

$$t(n) = n-1 + t(1) = n-1 + 1 = O(n)$$

Consider the following time complexity:

$$t(0) = 1$$

$$t(n) = 1 + 2t(n-1) = 1 + 2(1 + 2t(n-2)) = 1 + 2 + 4t(n-2) \\ = 1 + 2 + 4(1 + 2t(n-3)) = 1 + 2 + 4 + 8t(n-3) \\ = 1 + 2 + \dots + 2^{k-1} + 2^k t(n-k)$$

Choose $k = n$

$$t(n) = 1 + 2 + \dots + 2^{n-1} + 2^n = 2^{n+1} - 1$$

Binary Search

- Given a **sorted** vector/list `a[ ]`, find the location of element `X`

```
unsigned int binary_search(vector<int> a, int X)
{
    unsigned int low = 0, high = a.size()-1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] < X)
            low = mid + 1;
        else if( a[mid] > X )
            high = mid - 1;
        else
            return mid;
    }
    return NOT_FOUND;
}
```

- Input size: `n = array size()`
- Complexity = $O(k \text{ iterations} \times (1 \text{ comparison} + 1 \text{ assignment}) \text{ per loop})$
= $O(\log(n))$

Example

- Code:
- `a = b;`
- Complexity:

Example

- Code:

- ```
sum = 0;
```
- ```
for (i=1; i <=n; i++)
```
- ```
 sum += n;
```

- Complexity:

# Example

- **Code:**

- ```
sum = 0;
```
- ```
for (j=1; j<=n; j++)
```
- ```
    for (i=1; i<=j; i++)
```
- ```
 sum++;
```
- ```
for (k=0; k<n; k++)
```
- ```
 A[k] = k;
```

- **Complexity:**

# Example

- Code:
  - `sum1 = 0;`
  - `for (i=1; i<=n; i++)`
  - `for (j=1; j<=n; j++)`
  - `sum1++;`
- Complexity:

# Example

- Code:

- `sum2 = 0;`
- `for (i=1; i<=n; i++)`
- `for (j=1; j<=i; j++)`
- `sum2++;`

- Complexity:

# Example

- Code:
  - `sum1 = 0;`
  - `for (k=1; k<=n; k*=2)`
  - `for (j=1; j<=n; j++)`
  - `sum1++;`
- Complexity:

# Example

- Code:

- `sum2 = 0;`
- `for (k=1; k<=n; k*=2)`
- `for (j=1; j<=k; j++)`
- `sum2++;`

- Complexity:

# Algorithms for computing the Factorial

## Recursion

```
int factorial (int n)
{
 If (n <= 1)
 return 1;
 else
 return n * factorial(n-1);
}
```

## Iterative

```
int factorial (int n)
{
 fact =1;
 if (n<=1)
 return 1;
 else {
 for (k=2; k<=n; k++)
 fact *= k;
 return fact;
 }
}
```

# Iterative

```
Function()
{
 int I;
 for(i=1 to n)
 print("Hello");
}
```

$O(n)$

```
Function()
{
 int I;
 for(i=1 to n)
 for(j=1 to n)
 print("Hello");
}
```

$O(n^2)$



# Time complexity?

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
S
i
```

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
S
i 1
```

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
```

```
}
```

```
S
```

```
1
```

```
i
```

```
1
```

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
```

```
}
```

S

1

i

1

2

```

function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}

```

|   |   |   |
|---|---|---|
| S | 1 | 3 |
| i | 1 | 2 |

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

S

1

3

i

1

2

3

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

|   |   |   |   |
|---|---|---|---|
| S | 1 | 3 | 6 |
| i | 1 | 2 | 3 |

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

S

1

3

6

i

1

2

3

4



```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

|   |   |   |   |    |
|---|---|---|---|----|
| S | 1 | 3 | 6 | 10 |
| i | 1 | 2 | 3 | 4  |

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

|   |   |   |   |    |   |
|---|---|---|---|----|---|
| S | 1 | 3 | 6 | 10 |   |
| i | 1 | 2 | 3 | 4  | 5 |

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

|   |   |   |   |    |    |
|---|---|---|---|----|----|
| S | 1 | 3 | 6 | 10 | 15 |
| i | 1 | 2 | 3 | 4  | 5  |

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

S

1

3

6

10

15

i

1

2

3

4

5

6

```
function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}
```

|   |   |   |   |    |    |    |
|---|---|---|---|----|----|----|
| S | 1 | 3 | 6 | 10 | 15 | 21 |
| i | 1 | 2 | 3 | 4  | 5  | 6  |

```

function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}

```

|   |   |   |   |    |    |         |  |
|---|---|---|---|----|----|---------|--|
| S | 1 | 3 | 6 | 10 | 15 | 21      |  |
| i | 1 | 2 | 3 | 4  | 5  | 6.....n |  |

```

function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}

```

|   |   |   |   |    |    |         |            |          |
|---|---|---|---|----|----|---------|------------|----------|
| S | 1 | 3 | 6 | 10 | 15 | 21..... | $n(n+1)/2$ | n>s stop |
| i | 1 | 2 | 3 | 4  | 5  | 6.....  | n          |          |

```

function(){
 int i=1, s=1;
 while(S<=n)
 {
 i++;
 s=s+i;
 printf("hello")
 }
}

```

|   |   |   |   |    |    |                 |
|---|---|---|---|----|----|-----------------|
| S | 1 | 3 | 6 | 10 | 15 | 21.....k(k+1)/2 |
| i | 1 | 2 | 3 | 4  | 5  | 6.....k         |

$K(K+1)/2 \geq n$   
 $(K^2+k)/2 \geq n$

Which means  $K^2 = n$   
 $k = \sqrt{n}$

ie  $O(\sqrt{n})$



# Time complexity?

```
Function(){
 i=1;
 for(i=1;i2<=n;i++)
 printf("hello");
}
```

$$i^2 \leq n$$

or  $i = \sqrt{n}$

$O(\sqrt{n})$ , or theta  $\sqrt{n}$  both are same

# Time complexity?

```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}
```

I

J

K

```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}
```

I  
J  
K

1

```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}
```

|   |   |
|---|---|
| I | 1 |
| J | 1 |
| K |   |

```
Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}
```

|   |       |
|---|-------|
| I | 1     |
| J | 1     |
| K | 1*100 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |   |
|---|-------|---|
| I | 1     | 2 |
| J | 1     |   |
| K | 1*100 |   |

```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}
```

|   |       |   |
|---|-------|---|
| I | 1     | 2 |
| J | 1     | 2 |
| K | 1*100 |   |

```

Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |
|---|-------|-------|
| I | 1     | 2     |
| J | 1     | 2     |
| K | 1*100 | 2*100 |



```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |   |
|---|-------|-------|---|
| I | 1     | 2     | 3 |
| J | 1     | 2     |   |
| K | 1*100 | 2*100 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |   |
|---|-------|-------|---|
| I | 1     | 2     | 3 |
| J | 1     | 2     | 3 |
| K | 1*100 | 2*100 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |
|---|-------|-------|-------|
| I | 1     | 2     | 3     |
| J | 1     | 2     | 3     |
| K | 1*100 | 2*100 | 3*100 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |   |
|---|-------|-------|-------|---|
| I | 1     | 2     | 3     | 4 |
| J | 1     | 2     | 3     |   |
| K | 1*100 | 2*100 | 3*100 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |   |
|---|-------|-------|-------|---|
| I | 1     | 2     | 3     | 4 |
| J | 1     | 2     | 3     | 4 |
| K | 1*100 | 2*100 | 3*100 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |
|---|-------|-------|-------|-------|
| I | 1     | 2     | 3     | 4     |
| J | 1     | 2     | 3     | 4     |
| K | 1*100 | 2*100 | 3*100 | 4*100 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |   |
|---|-------|-------|-------|-------|---|
| I | 1     | 2     | 3     | 4     | 5 |
| J | 1     | 2     | 3     | 4     |   |
| K | 1*100 | 2*100 | 3*100 | 4*100 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |   |
|---|-------|-------|-------|-------|---|
| I | 1     | 2     | 3     | 4     | 5 |
| J | 1     | 2     | 3     | 4     | 5 |
| K | 1*100 | 2*100 | 3*100 | 4*100 |   |



```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |       |
|---|-------|-------|-------|-------|-------|
| I | 1     | 2     | 3     | 4     | 5     |
| J | 1     | 2     | 3     | 4     | 5     |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |         |
|---|-------|-------|-------|-------|---------|
| I | 1     | 2     | 3     | 4     | 5.....n |
| J | 1     | 2     | 3     | 4     | 5       |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |         |
|---|-------|-------|-------|-------|---------|
| I | 1     | 2     | 3     | 4     | 5.....n |
| J | 1     | 2     | 3     | 4     | 5.....n |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |                 |
|---|-------|-------|-------|-------|-----------------|
| I | 1     | 2     | 3     | 4     | 5.....n         |
| J | 1     | 2     | 3     | 4     | 5.....n         |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100.....n*100 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |                 |
|---|-------|-------|-------|-------|-----------------|
| I | 1     | 2     | 3     | 4     | 5.....n         |
| J | 1     | 2     | 3     | 4     | 5.....n         |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100.....n*100 |

$1*100+2*100+3*100+4*100+.....n*100$

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |                 |
|---|-------|-------|-------|-------|-----------------|
| I | 1     | 2     | 3     | 4     | 5.....n         |
| J | 1     | 2     | 3     | 4     | 5.....n         |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100.....n*100 |

$1*100+2*100+3*100+4*100+.....n*100$   
 $100(1+2+3+4+.....n)$

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<=i;j++){
 for(k=1;k<=100;k++){
 print("Hello");
 }
 }
 }
}

```

|   |       |       |       |       |             |
|---|-------|-------|-------|-------|-------------|
| I | 1     | 2     | 3     | 4     | 5.....n     |
| J | 1     | 2     | 3     | 4     | 5.....n     |
| K | 1*100 | 2*100 | 3*100 | 4*100 | 5*100.....n |

$1*100+2*100+3*100+4*100+.....n*100$

$100(1+2+3+4+.....n)$        $n(n+1)/2$  ie  $O(n^2)$

# Time complexity?

```
Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}
```

i  
j  
k



```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}
```

I  
J  
K

1

```
Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}
```

|   |   |
|---|---|
| I | 1 |
| J | 1 |
| K |   |

```

Function(){
 int I,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |
|---|-------|
| I | 1     |
| J | 1     |
| K | 1*n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |   |
|---|-------|---|
| I | 1     | 2 |
| J | 1     |   |
| K | 1*n/2 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |   |
|---|-------|---|
| I | 1     | 2 |
| J | 1     | 4 |
| K | 1*n/2 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |
|---|-------|-------|
| I | 1     | 2     |
| J | 1     | 4     |
| K | 1*n/2 | 4*n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |   |
|---|-------|-------|---|
| I | 1     | 2     | 3 |
| J | 1     | 4     |   |
| K | 1*n/2 | 4*n/2 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |   |
|---|-------|-------|---|
| I | 1     | 2     | 3 |
| J | 1     | 4     | 9 |
| K | 1*n/2 | 4*n/2 |   |



```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |               |               |               |
|---|---------------|---------------|---------------|
| I | 1             | 2             | 3             |
| J | 1             | 4             | 9             |
| K | $1 \cdot n/2$ | $4 \cdot n/2$ | $9 \cdot n/2$ |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |   |
|---|-------|-------|-------|---|
| I | 1     | 2     | 3     | 4 |
| J | 1     | 4     | 9     |   |
| K | 1*n/2 | 4*n/2 | 9*n/2 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |    |
|---|-------|-------|-------|----|
| I | 1     | 2     | 3     | 4  |
| J | 1     | 4     | 9     | 16 |
| K | 1*n/2 | 4*n/2 | 9*n/2 |    |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |
|---|-------|-------|-------|--------|
| I | 1     | 2     | 3     | 4      |
| J | 1     | 4     | 9     | 16     |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |   |
|---|-------|-------|-------|--------|---|
| I | 1     | 2     | 3     | 4      | 5 |
| J | 1     | 4     | 9     | 16     |   |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 |   |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |        |
|---|-------|-------|-------|--------|--------|
| I | 1     | 2     | 3     | 4      | 5      |
| J | 1     | 4     | 9     | 16     | 25     |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |        |
|---|-------|-------|-------|--------|--------|
| I | 1     | 2     | 3     | 4      | 5      |
| J | 1     | 4     | 9     | 16     | 25     |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |         |
|---|-------|-------|-------|--------|---------|
| I | 1     | 2     | 3     | 4      | 5.....n |
| J | 1     | 4     | 9     | 16     | 25      |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2  |



```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |                       |
|---|-------|-------|-------|--------|-----------------------|
| I | 1     | 2     | 3     | 4      | 5.....n               |
| J | 1     | 4     | 9     | 16     | 25.....n <sup>2</sup> |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2                |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |                                |
|---|-------|-------|-------|--------|--------------------------------|
| I | 1     | 2     | 3     | 4      | 5.....n                        |
| J | 1     | 4     | 9     | 16     | 25.....n <sup>2</sup>          |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2.....n <sup>2</sup> *n/2 |

```

Function(){
 int l,j,k,n;
 for(i=1;i<=n;i++){
 for(j=1;j<= i2;j++){
 for(k=1;k<=n/2;k++){
 printf("Hello");
 }
 }
 }
}

```

|   |       |       |       |        |                                |
|---|-------|-------|-------|--------|--------------------------------|
| I | 1     | 2     | 3     | 4      | 5.....n                        |
| J | 1     | 4     | 9     | 16     | 25..... n <sup>2</sup>         |
| K | 1*n/2 | 4*n/2 | 9*n/2 | 16*n/2 | 25*n/2.....n <sup>2</sup> *n/2 |

1\*n/2    4\*n/2    9\*n/2    16\*n/2    25\*n/2.....n<sup>2</sup>\*n/2

$n/2(1^2 + 2^2 + 3^2 + \dots + n^2)$

$n/2((n(n+1)(2n+1)/6)$

$O(n^4)$

# Time complexity?

```
Function(){
 for(i=1;i<n;i=i*2)
 printf("hello");
}
```

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| i=1   | 2     | 4     | 8     | 16    | n     |
| $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | $2^k$ |

$$2^k = n$$

$$K = \log(n)$$

$$O(\log_2 n)$$

# Time complexity?

```
Function(){
 int i,j,k;
 for(i=n/2;i<=n;i++)
 for(j=1;j<=n/2;j++)
 for(k=1;k<=n;k=k*2)
 printf("hello");
}
```

```

Function(){
 int i,j,k;
 for(i=n/2;i<=n;i++)
 for(j=1;j<=n/2;j++)
 for(k=1;k<=n;k=k*2)
 printf("hello");
}

```

$n/2 * n/2 * \log_2 n$

$O(n^2 \log_2 n)$

```
Function(){
 int l,j,k;
 for(i=n/2;i<=n;i++)
 for(j=1;j<=n;j=2*j)
 for(k=1;k<=n;k=k*2)
 printf("Hello");
}
```

```

Function(){
 int l,j,k;
 for(i=n/2;i<=n;i++)
 for(j=1;j<=n;j=2*j)
 for(k=1;k<=n;k=k*2)
 printf("Hello");
}

```

$n/2 * (\log_2 n)^2$

$O(n * (\log_2 n)^2)$



# Time complexity?

```
Function(){
 for(i=1;i<=n,i++)
 for(j=1;j<=n;j=j+i)
 printf("hello");
}
```

I

J

```
Function(){
 for(i=1;i<=n,i++)
 for(j=1;j<=n;j=j+i)
 printf("hello");
}
```

i=1    2    3    4

J=n    n/2    n/3    n/4

$n + n/2 + n/3 + \dots + 1$ )

$n(1 + 1/2 + 1/3 + 1/4 + \dots + 1/n)$

$n(\log n)$

# Recursive Functions

```
Function(n){
 if(n>1)
 return(function(n-1))
}
```

```
Function(n){
 if(n>1)
 return(function(n-1))
}
```

T(1)

```
Function(n){
```

```
 if(n>1)
```

```
 return(function(n-1))
```

```
}
```

$T(1)$

$T(n-1)$

```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$T(n)=1+T(n-1)$       eq(1)

```

Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}

```

$$T(n) = 1 + T(n-1); \quad n > 1 \quad \text{eq(1)}$$

$$T(1) = 1 + T(1-1); \quad n = 1$$

$$T(1) = 1 + T(0);$$

$$T(1) = 1$$

$$T(n) = 1 + T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$$T(n)=1+T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1)=1+T(n-2) \quad \text{eq(2)}$$



```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$$T(n)=1+T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1)=1+T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$$T(n)=1+T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1)=1+T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2)=1+T(n-3) \quad \text{eq(3)}$$

```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$$T(n)=1+T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1)=1+T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2)=1+T(n-3) \quad \text{eq(3)}$$

Backward substitution

```

Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}

```

$$T(n) = 1 + T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1) = 1 + T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2) = 1 + T(n-3) \quad \text{eq(3)}$$

Backward substitution

Substitute eq(2) in 1  $T(n) = 1 + (1 + T(n-2)) = 2 + T(n-2)$

```

Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}

```

$$T(n) = 1 + T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1) = 1 + T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2) = 1 + T(n-3) \quad \text{eq(3)}$$

Backward substitution

$$\text{Substitute eq(2) in 1} \quad T(n) = 1 + (1 + T(n-2)) = 2 + T(n-2)$$

$$\text{Substitute eq(3) in 2} \quad T(n) = 2 + (1 + T(n-3)) = 3 + T(n-3)$$

```

Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}

```

$$T(n) = 1 + T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1) = 1 + T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2) = 1 + T(n-3) \quad \text{eq(3)}$$

Backward substitution

$$\text{Substitute eq(2) in 1} \quad T(n) = 1 + (1 + T(n-2)) = 2 + T(n-2)$$

$$\text{Substitute eq(3) in 2} \quad T(n) = 2 + (1 + T(n-3)) = 3 + T(n-3)$$

$$K + T(n-k) \quad \text{eq(4)}$$

```
Function(n){
 if(n>1)
 return(function(n-1))
}
```

$$T(n)=1+T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1)=1+T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2)=1+T(n-3) \quad \text{eq(3)}$$

Backward substitution

$$\text{Substitute eq(2) in 1 } T(n) = 1+(1+T(n-2)) = 2+T(n-2)$$

$$\text{Substitute eq(3) in 2 } T(n)=2+(1+T(n-3)) = 3+T(n-3)$$

$$K+T(n-k) \quad \text{eq(4)}$$

When  $n-k=1$  loop stop so  $k=n-1$  substitute in eq(4)

```
Function(n){
 if(n>1)
 return(function(n-1))
}
```

$T(1)$

$T(n-1)$

$T(n)=1+T(n-1)$  eq(1)

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$T(n-1)=1+T(n-2)$  eq(2)

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$T(n-2)=1+T(n-3)$  eq(3)

Backward substitution

Substitute eq(2) in 1  $T(n) = 1+(1+T(n-2)) = 2+T(n-2)$

Substitute eq(3) in 2  $T(n)=2+(1+T(n-3)) = 3+T(n-3)$

$K+T(n-k)$

When  $n-k=1$  loop stop so  $k=n-1$  substitute in eq(4)

$n-1+T(n-(n-1))$



```
Function(n){
 if(n>1) T(1)
 return(function(n-1)) T(n-1)
}
```

$T(n)=1+T(n-1)$  eq(1)

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$T(n-1)=1+T(n-2)$  eq(2)

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$T(n-2)=1+T(n-3)$  eq(3)

Backward substitution

Substitute eq(2) in 1  $T(n) = 1+(1+T(n-2)) = 2+T(n-2)$

Substitute eq(3) in 2  $T(n)=2+(1+T(n-3)) = 3+T(n-3)$

$K+T(n-k)$

When  $n-k=1$  loop stop so  $k=n-1$  substitute in eq(4)

$n-1+T(n-(n-1))$

$n$

```
Function(n){
 if(n>1)
 return(function(n-1))
}
```

$T(1)$

$T(n-1)$

$T(n)=1+T(n-1)$   $eq(1)$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in  $eq(1)$

$T(n-1)=1+T(n-2)$   $eq(2)$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in  $eq(1)$

$T(n-2)=1+T(n-3)$   $eq(3)$

Backward substitution

Substitute  $eq(2)$  in 1  $T(n) = 1+(1+T(n-2)) = 2+T(n-2)$

Substitute  $eq(3)$  in 2  $T(n)=2+(1+T(n-3)) = 3+T(n-3)$

$K+T(n-k)$

When  $n-k=1$  loop stop so  $k=n-1$  substitute in  $eq(4)$

$n-1+T(n-(n-1))$

$n$

$O(n)$

# Problem II

$$T(n) = n + T(n-1) \quad n > 1$$

$$T(1) = 1 + T(1-1) \quad n = 1$$

$$T(n) = n + T(n-1) \quad \text{eq(1)}$$

To find  $T(n-1)$  substitute  $n-1$  to all  $n$  in eq(1)

$$T(n-1) = n-1 + T(n-2) \quad \text{eq(2)}$$

To find  $T(n-2)$  substitute  $n-2$  to all  $n$  in eq(1)

$$T(n-2) = n-2 + T(n-3) \quad \text{eq(3)}$$

Backward substitution

$$T(n) = n + T(n-1)$$

Substitute eq(2) in 1  $T(n) = n + (n-1) + T(n-2)$

Substitute eq(3) in 2  $T(n) = n + (n-1) + (n-2) + T(n-3) \dots + (n-k) + \underline{T(n-(k+1))}$  eq(4)



When  $n-(k+1)=1$  loop stop so  $k=n-2$  substitute in eq(4)

$$n-(K+1) = 1$$

$$n-k-1=1$$

$$K=n-2$$

$$T(1)$$

$$T(n) = n + (n-1) + (n-2) + T(n-3) \dots + (n - (n-2)) + T(n - ((n-2) + 1))$$

$$= n + (n-1) + (n-2) + T(n-3) \dots + 2 + 1$$

$$= n(n+1)/2$$

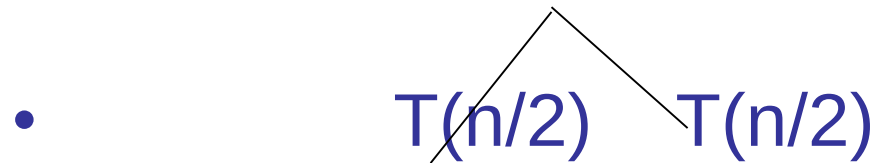
$$= O(n^2)$$

# Recursion Tree Method

## Problem 1

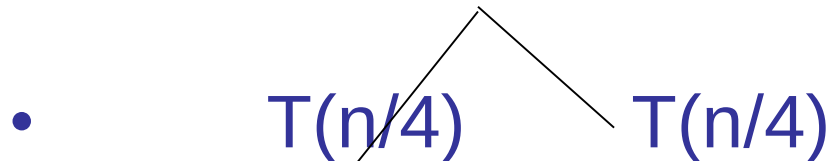
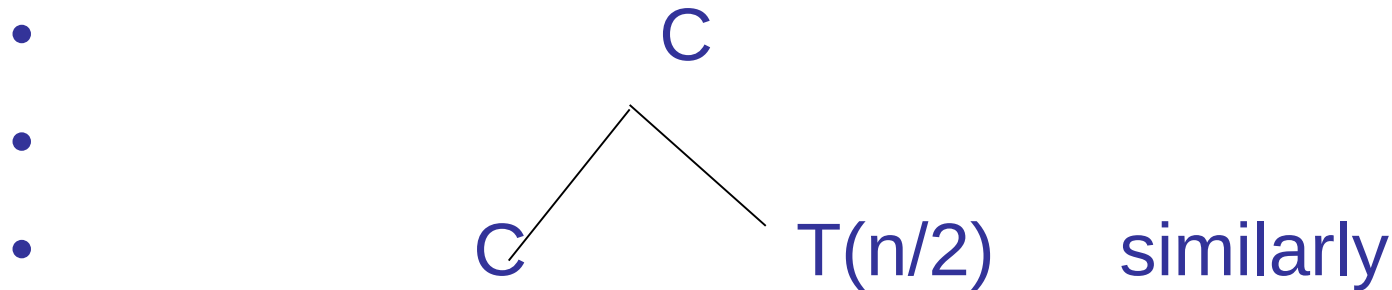
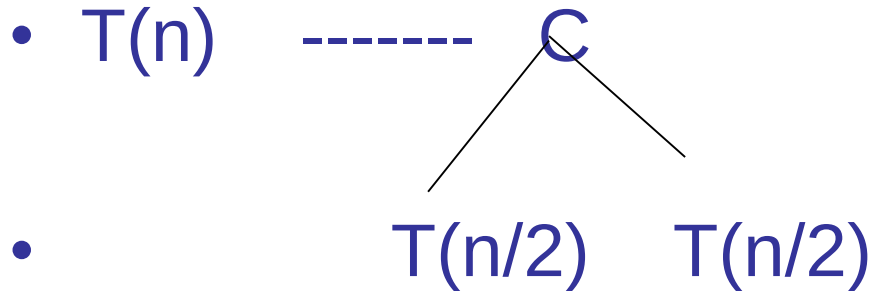
- $T(n) = 2T(n/2) + C$  ;  $n > 1$
- $= C$  ;  $n = 1$
- if I do  $C$  amount of work, I could produce 2 problems with smaller sets of inputs and each input size is reduced to  $n/2$ .

- $T(n)$  -----  $C$

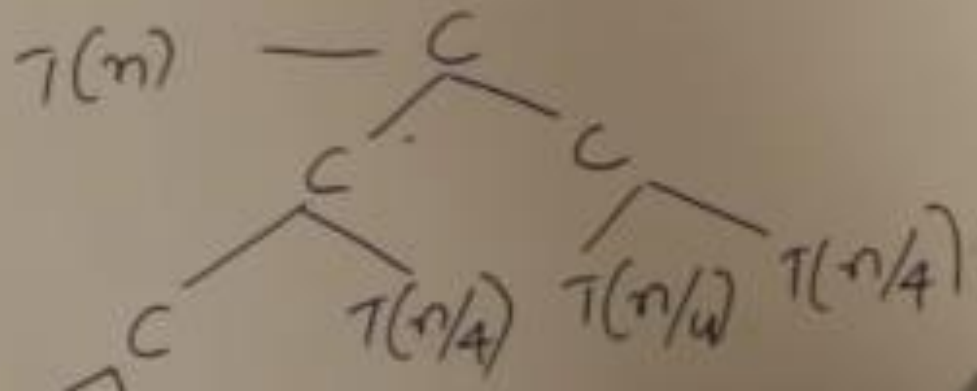


## Problem I

- $T(n) = 2T(n/2) + C$  ;  $n > 1$   $t(n/2) = 2t(n/4) + c$
- $= C$  ;  $n = 1$   $t(n/4) = 2t(n/8) + c$

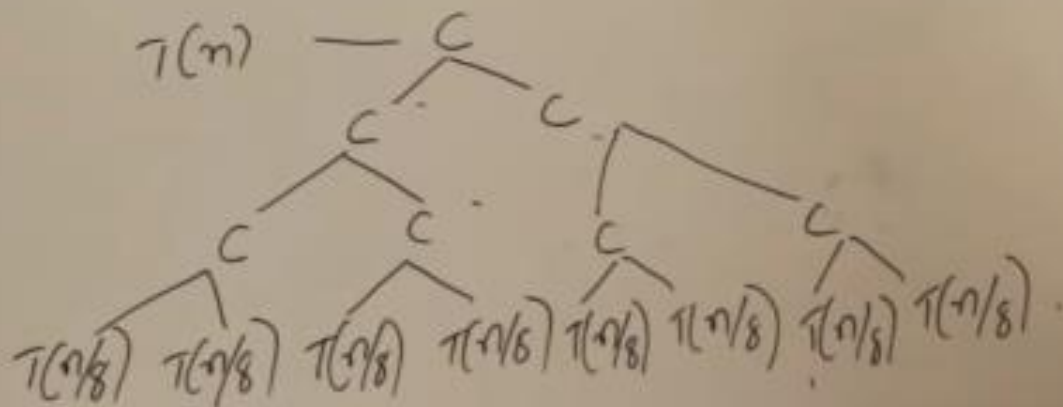


## Problem I



Recursion tree method:

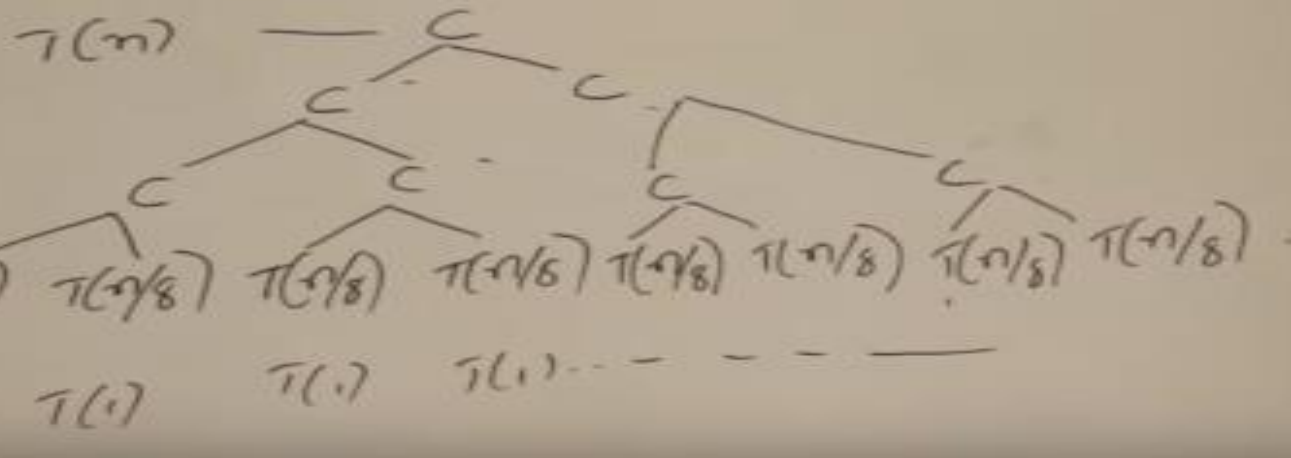
$$T(n) = 2T(n/2) + C; n > 1$$
$$= C; n = 1$$



Recursion tree method :

$$T(n) = 2T(n/2) + C ; n > 1$$

$$= C ; n = 1$$



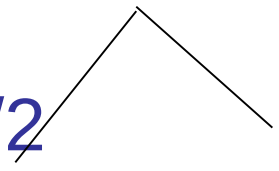
C  
2C  
4C  
8C

nC

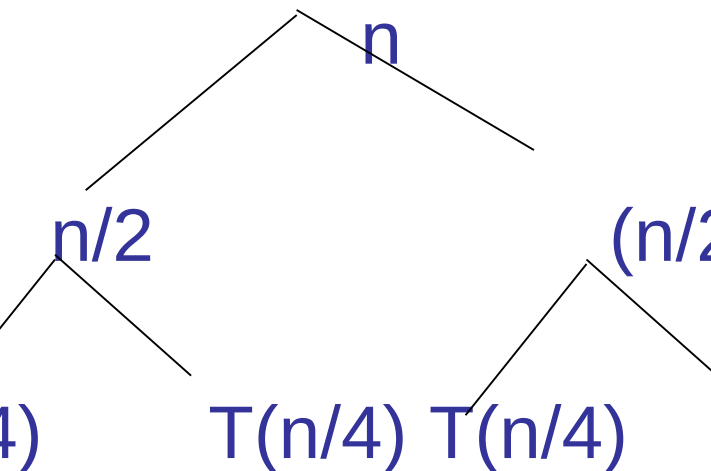
- $T(1) = T(n/n)$
- $C + 2C + 4C + \dots nC$
- $C(1 + 2 + 4 + \dots n)$  assume  $n = 2^k$  (for simplification)
- $C(1 + 2 + 4 + \dots 2^k) \Rightarrow C(1(2^{k+1} - 1)) / (2 - 1)$
- $\Rightarrow C(2^{k+1} - 1) \Rightarrow C(2n - 1)$
- $\Rightarrow O(n)$

- $T(n) = 2T(n/2) + n$  ;  $n > 1$
- $= 1$  ;  $n = 1$

- $T(n)$  -----  $n$

-   $T(n/2) =$

-   $T(n/4) =$

-   $(n/2)$  similarly
- $T(n/4)$   $T(n/4)$   $T(n/4)$   $T(n/4)$

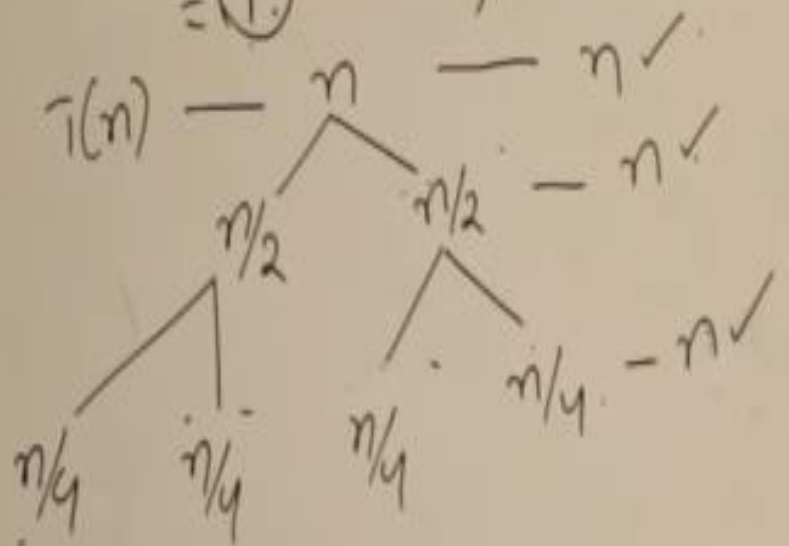


Recursion tree method:

$$T(n) = 2T(n/2) + n; n > 1$$

= ①

; n = 1



$$\begin{aligned} T(n) &= 2T(n/2) + cn && ; n > 1 \\ &= 1 && ; n = 1 \end{aligned}$$